ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PAPER 1 · PRACTICE E · MARKING GUIDELINE

MARKS: 120

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Technical Programming paper 1 · practice e.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Code is not marked strictly on syntax: give marks for correct logic and structure. Accept other correct answers, other variable names and other correct flowchart layouts.

SECTION A

QUESTION 1

1.1 B ✓
1.2 D ✓
1.3 A ✓
1.4 C ✓
1.5 A ✓
1.6 B ✓
1.7 C ✓
1.8 D ✓
1.9 A ✓
1.10 B ✓

(10)

[10]

QUESTION 2

2.1 True ✓
2.2 True ✓
2.3 False ✓
2.4 True ✓
2.5 False ✓

(5)

[5]

QUESTION 3

3.1 C ✓
3.2 G ✓
3.3 B ✓
3.4 F ✓
3.5 J ✓
3.6 I ✓
3.7 D ✓
3.8 H ✓
3.9 A ✓
3.10 E ✓

(10)

[10]

QUESTION 4

4.1 Initialisation (the counter is declared and set to 0) ✓

4.2 Condition (test) ✓

4.3 Update (increment) ✓

4.4 Start of the loop body ✓

4.5 Statement in the loop body (output) ✓

4.6 End of the loop body ✓

(6)

[6]

TOTAL SECTION A: [31]

SECTION B

QUESTION 5

5.1.1 A software package that combines the tools to write, compile, run and debug programs ✓ in one place (editor, compiler, debugger) ✓ (2)

5.1.2 The kind of value a variable can store and the operations allowed on it ✓✓. Examples ✓✓: int, double, char, string, bool. (4)

5.1.3 ROM (or flash ROM) ✓✓ (2)

5.1.4 ALU (Arithmetic Logic Unit) ✓✓ (2)

5.1.5 A program that translates the whole source code into machine code ✓ before the program is run ✓. (2)

5.2 Looking for similarities or patterns in problems or data ✓, so that a solution can be reused ✓. (2)

5.3 A for loop is used when the number of repetitions is known, and its counter, condition and update are written in one line ✓✓. A while loop repeats while a condition is true, when the number of repetitions is not known in advance ✓✓. (4)

5.4 A table with a column for each variable (and output) where the values are written down line by line as the program runs ✓✓; it is used to check that the logic is correct and to find errors ✓✓. (4)

5.5 Any FOUR ✓✓✓✓: must start with a letter or underscore; no spaces; only letters, digits and underscores; cannot be a C++ keyword (e.g. int, for); names are case sensitive; should be meaningful. (4)

[26]

QUESTION 6

Any ELEVEN relevant points ✓ (one mark each), e.g.: Decomposition ✓ – break the job into smaller parts: detect an item, identify the material, move the arm, drop in the correct bin, count items ✓✓. Pattern recognition ✓ – the same steps repeat for every item; materials have patterns (metal triggers the sensor, glass is transparent, paper is flat and light) ✓✓. Abstraction ✓ – focus only on the details needed to sort (material type, position) and ignore others (colour of labels, brand names) ✓✓. Algorithm design ✓ – write the step-by-step solution (pseudocode / flowchart): wait for an item, read the sensors, decide the material, move the arm to that bin, repeat ✓✓. (11)

[11]

QUESTION 7

7.1 Any FOUR ✓✓✓✓: total – sum of the temperatures; count – number of temperatures entered; highest – largest temperature so far; temp – the temperature just entered; average – total ÷ count. (4)

7.2 It adds the temperature just entered to the running total ✓✓. (2)

7.3 It gives a starting value that any real temperature will beat, so the first temperature becomes the highest ✓✓; if all temperatures are below 0 (negative, e.g. in a freezer), highest would wrongly stay 0 ✓. (3)

7.4 Total = 125 ✓; average = 125 ÷ 5 = 25 ✓; 25 is not more than 25, so "Cool week" ✓; "Highest: 31" ✓ (4)

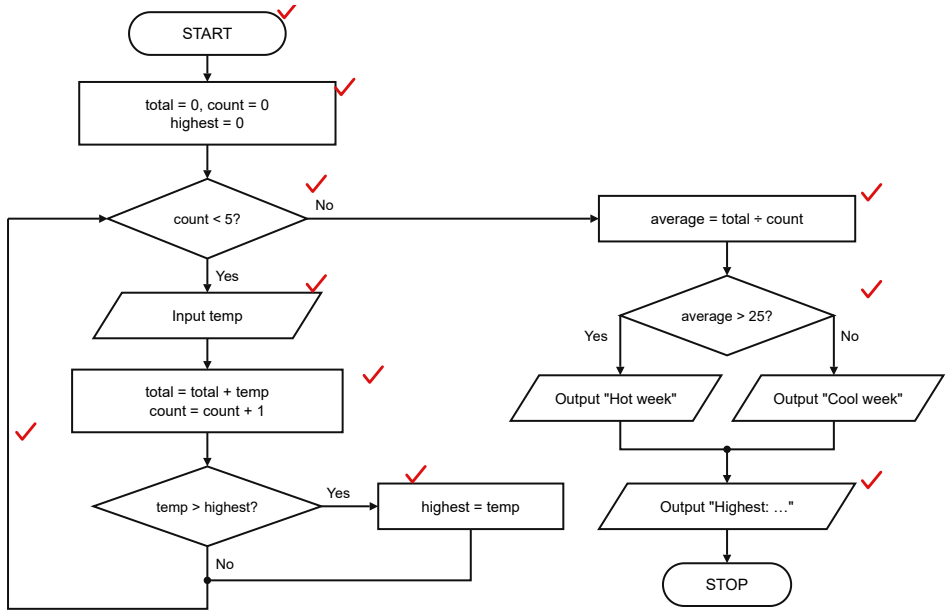
7.5 Libraries and main() ✓; variables set to 0 ✓; for loop that runs 5 times ✓; input of temp ✓; total and count updated ✓; highest updated with an if ✓; average calculated ✓; correct messages ✓, e.g. (8)

```
#include <iostream>
using namespace std;

int main() {
    double temp, total = 0, highest = 0, average;
    int count = 0;
    for (int day = 1; day <= 5; day++) {
        cout << "Enter today's temperature: ";
        cin >> temp;
        total += temp;
        count++;
        if (temp > highest) highest = temp;
    }
    average = total / count;
    if (average > 25) cout << "Hot week" << endl;
    else cout << "Cool week" << endl;
    cout << "Highest: " << highest << endl;
    return 0;
}
```

7.6 The number of repetitions (5 days) is known in advance ✓, and a for loop keeps the counter, condition and update together ✓. (2)

7.7



(Start 1; total, count and highest set to 0 1; loop decision count < 5 1; input temp 1; total = total + temp, count + 1 1; decision temp > highest → highest = temp 1; after the loop average = total ÷ count 1; decision average > 25 → "Hot week" / "Cool week" 1; output highest and stop 1)

(9)

[32]

QUESTION 8

8.1 Five elements each ✓; the values are given in an initialiser list in curly braces when the arrays are declared ✓. (2)

8.2 2 ✓✓ (counting starts at 0) (2)

8.3 battery is out of stock ✓✓ (2)

8.4 Only 4 left - reorder soon ✓✓ (2)

8.5 7 in stock ✓✓ (2)

8.6 Unknown part ✓✓ (2)

8.7 It records whether the part was found in the array ✓, so "Unknown part" is shown only when no match was found ✓. (2)

8.8 It is the logical NOT: it reverses the value, so !found is true when found is false ✓✓. (2)

8.9 It provides the string data type used for the part names and the input ✓✓. (2)

8.10 Add a break statement after found = true / after the output ✓✓ (or use a while loop that stops when found is true). (2)

[20]

TOTAL SECTION B: [89]

TOTAL: 120

END OF MARKING GUIDELINE